NTNU
Department of Engineering Cybernetics

TTK4147 Real-time Systems

04 Scheduling

10.09.2025

1

Today

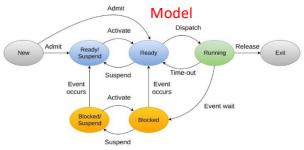
- Scheduling
  - Different Time-Scales
  - Scheduling Criteria
  - Priority Based Scheduling
  - Different Scheduling Policies
- Real-time Scheduling
  - Cyclic Executive
  - Fixed Priority Scheduling
  - Rate Monotonic
  - Is Scheduling Possible?
  - Response-time Analysis

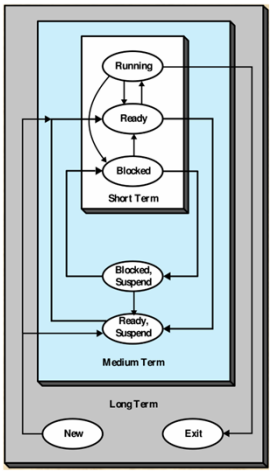
2

What is Scheduling?

- A strategy for deciding which process that are allowed to run on a processor over time.
  - Could also be scheduling of other resources, e.g. I/O resources.
- Common to break it down into three separate timescales:
  - Long-term scheduling is the decision of which processes that are allowed to be added to the pool of processes to be handled.
  - Medium-term scheduling is to decide which processes that are fully or partially swapped in to main memory.
  - Short-term scheduling is to decide which of the currently ready processes that the CPU should execute.

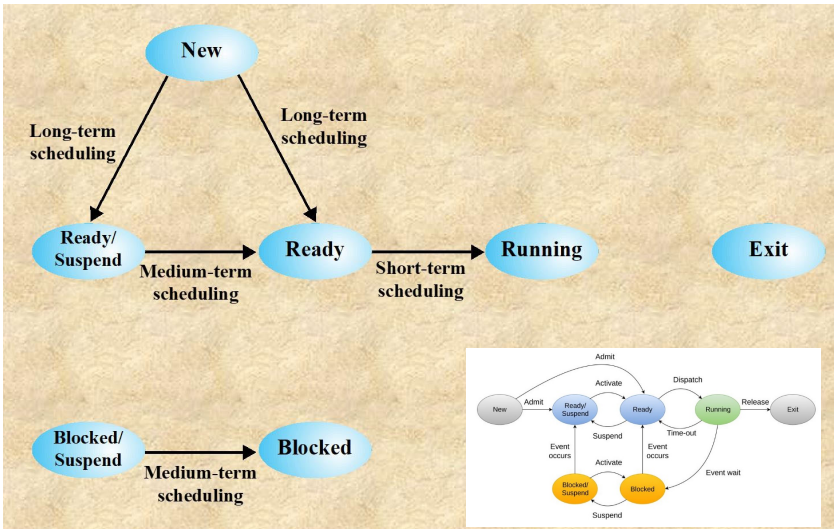
Seven-state Process Model





3

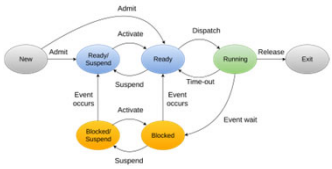
Scheduling and Process State transistions

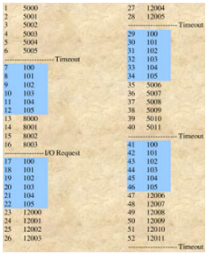


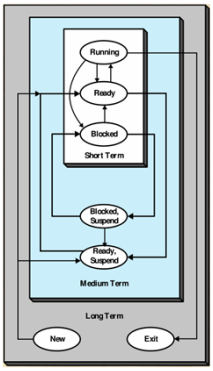
4

Short-Term Scheduling

- Main focus on this lecture*.
- The short-term scheduler is also called *dispatcher.*
- Main objective is to allocate processor time to optimize certain aspects (criteria**) of system behavior







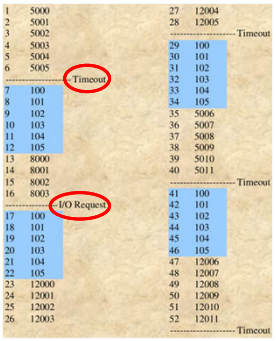
* When there is no room for a process (or the complete process) in main memory (RAM), it can be (partly) swapped to disk.
Still a part of the running process in virtual memory

** Which criteria are to be optimized?

5

Dispatcher

- Each time the dispatcher is called, it should evaluate which task is the next to run.
- The dispatcher can be called due to four reasons:
  - A clock interrupt which decides that the allowable time slice for the process has been used.
  - I/O interrupt means that the OS has to reevaluate which process should run.
  - A trap (error) will exit the current process.
  - The process perform an action that will cause it to wait/be blocked. (E.g. make a sleep call, Kernel calls)



6

Scheduling Criteria, User Oriented

- Performance related / quantitative
  - Turnaround time
    This is the interval of time between the submission of a process and its completion.
  - Waiting time
    The extra time spent for waiting; Turnaround time – Service time
  - Response time
    The time from the submission of a request until the start of response. [Stallings]
  - Deadlines
    When process completion deadlines can be specified, as many deadlines as possible should be met.
- Non-performance related / qualitative
  - Predictability
    A given job should run in about the same amount of time and at about the same cost regardless of the load on the system.

7

Scheduling Criteria, System Oriented

- Performance related / quantitative
  - Throughput
    Maximize the number of processes completed per unit of time. This is a measure of how much work is being performed.
  - Processor utilization
    This is the percentage of time that the processor is busy.
- Non-performance related / qualitative
  - Fairness
    Processes should be treated the same, and no process should suffer starvation.
  - Enforcing priorities
    When processes are assigned priorities, the scheduling policy should favor higher-priority processes.
  - Balancing resources
    Balanced use of system resources. If a resource is overused, then processes that do not need it should be favored.

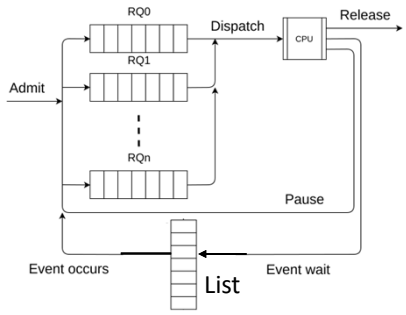
8

Different Scheduling Policies

9

Priority Based Scheduling

- Assign priorities to each process.
  - Dynamic or static.
- Each priority level gets its own queue, and the dispatcher will check higher priority queues first.
- Lower priority tasks might suffer from "starvation".



10

Nonpreemptive versus Preemptive

Nonpreemptive

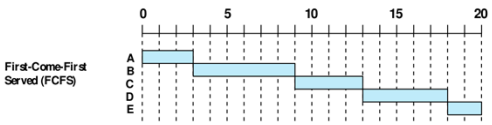
- Once a process is in the running state, it will continue until it terminates or block itself for I/O
  (run to completion)
- Favor long processes.

Preemptive

- Currently running process may be interrupted and moved to ready state by the OS
- Preemption may occur when a new process arrives, on an interrupt, at a system / kernel call or periodically
- Favor short processes.

First-Come-First-Served
(First-In-First-Out)

- Dispatcher chooses the process that has been in the ready queue the longest.
- Non-preemptive, run-to-completion
- Favor long processes.
- Favor processes that mostly use CPU over processes that mostly use I/O.
- Often used in combination with priorities. (One queue pr. priority level.)



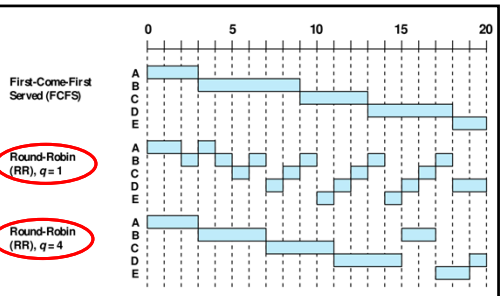
Why?

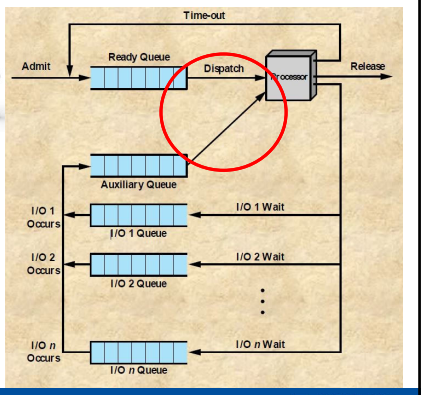
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

(Service Time = Time = Execution Time)

Round Robin

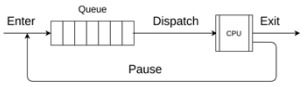
- Each process is given a time slice.
  - Regular clock interrupts (Pre-emptive).
  - Place running process in ready queue
  - Process that has been the longest in the ready queue is executed next.
- Preemptive
- Favor short processes.
- Favor processes that mostly use CPU over processes that mostly use I/O.
- Variations can mitigate this problem.
  - Virtual Round-robin.
  - Different strategies for how to handle Ready Queue versus Auxiliary Queue





13

How processes are queued up



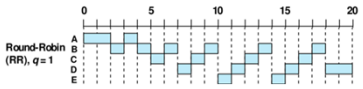
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Time	Queue	Running:
0	A	A
1	A	A
2	AB	B
3	BA	A
4	CB	B
5	BC	C
6	CDB	B
7	BCD	D
8	DEBC	C
9	CDEB	B
10	BCDE	E
11	EBCD	D
12	DEBC	C
13	CDEB	B
14	BCDE	E
15	BCD	D
16	DBC	C
17	DB	B
18	D	D
19	D	D

Swapped out after new

Time	Queue	Running:
0	A	A
1	A	A
2	BA	A
3	B	B
4	CB	B
5	BC	C
6	DCB	B
7	BDC	C
8	CEBD	D
9	DCEB	B
10	BDCE	E
11	EBDC	C
12	CEBD	D
13	DCEB	B
14	BDCE	E
15	BDC	C
16	BD	B
17	D	D
18	D	D
19	D	D

Swapped out before new



Swapped out after new process

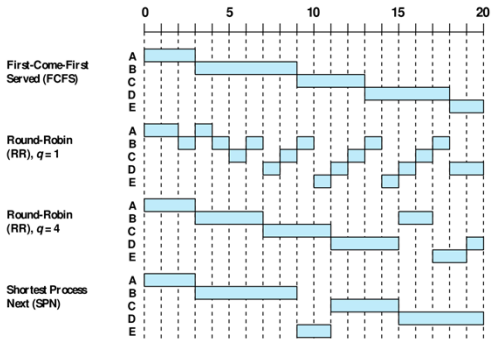
At t = 2, 4, 6, 8 swapping and arrival of new processes happen simultaneously

Swapped out after new process is commonly used.
(If you are using other definitions you must declare that.)

14

Shortest Process Next

- Dispatcher chooses the process in the queue with the shortest executing time.
- Non-preemptive
- Requires knowledge of executing time.
  ???
- Favor short processes.
- Long processes might never get a chance to run (starvation).

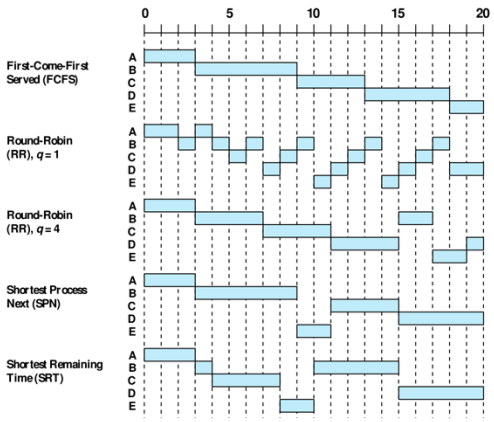


Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

15

Shortest Remaining Time

- Preemptive version of Shortes Process Next.
- At each tick (rescheduling), evaluates which task has the shortest remaining time.
- Requires knowledge of executing time.
  ???
- Favor short processes.
- Long processes might never get a chance to run (starvation).



Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

16

Highest Response Ratio Next

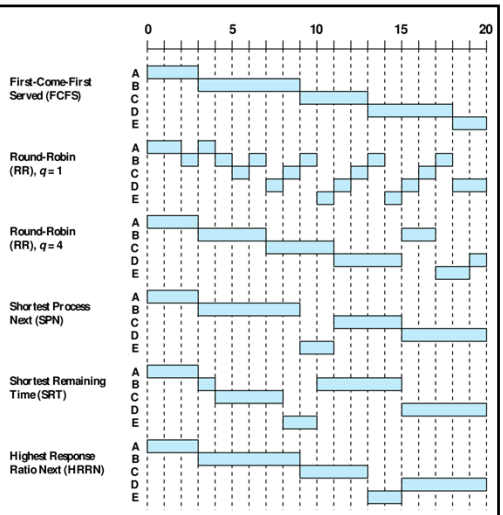
- Non-preemptive
- Dispatcher choose process from execution time and time waited in queue.
  - Ratio calculated for each process.

$$R = \frac{w + s}{s}$$

R = response ratio
w = time waited in queue
s = expected service time

  - Highest R get dispatched.
- Requires knowledge of executing time (service time).

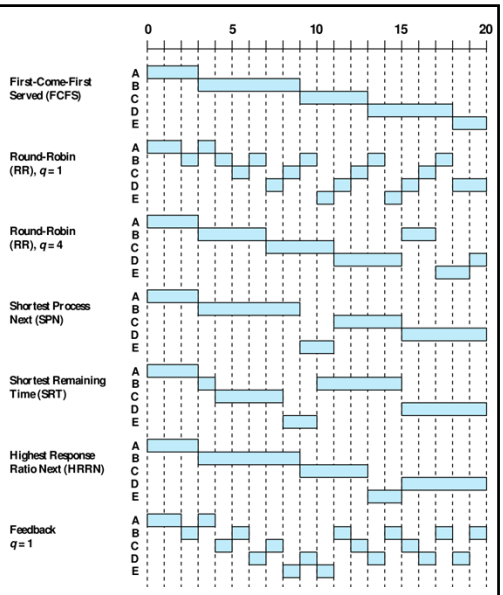
  ???
- Good balance.



17

Feedback

- Decrease priority each time process is preempted. (Preemptive)
- Newer and shorter processes favored over old and long.
- Does not require knowledge of execution time.
- Could cause starvation of long processes.
- Starvation can be mitigated by allowing longer time slices for lower-priority processes.



18

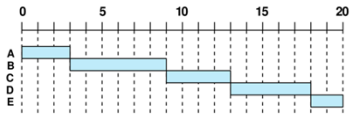
Process turnaround time (TaT), waiting time (WT)

(Scheduling Criteria, User Oriented)

Calculation of TaT and WT

TaT = Finish time – Arrival time

WT = Turnaround time – Service time



First come, first served

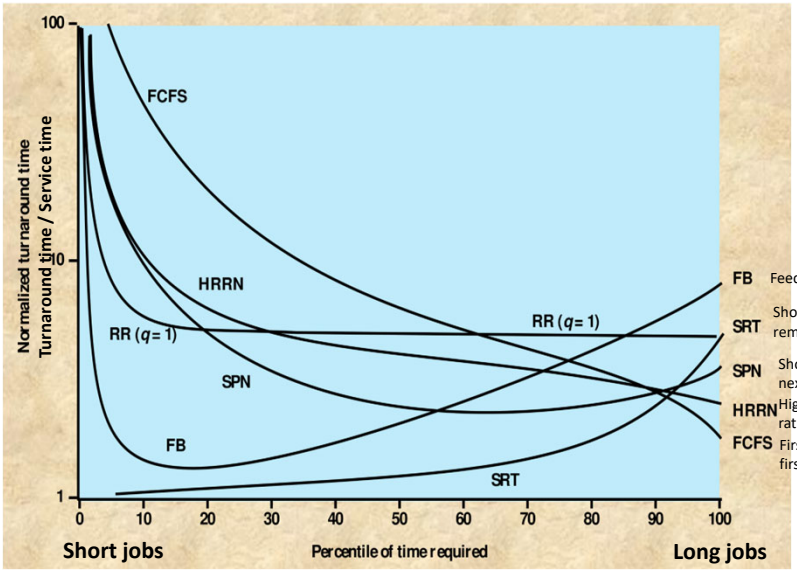
Process	Arrival time	Service time	Finish time	Turnaround time	Waiting time
A	0	3	3	3	0
B	2	6	9	7	1
C	4	4	13	9	5
D	6	5	18	12	7
E	8	2	20	12	10
			Mean:	43/5 = 8,6	23/5 =4,6

Mean process turnaround time and waiting time

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Scheduler	Mean TaT	Mean WT
First-Come-First-Served	8,6	4,6
Round-Robin, q=1	10,8	6,8
Round-Robin, q=4	10	6
Shortest Process Next	7,6	3,6
Shortest Remaining Time	7,2	3,2
Highest Response Ration Next	8	4
Feedback, q=1	10	6

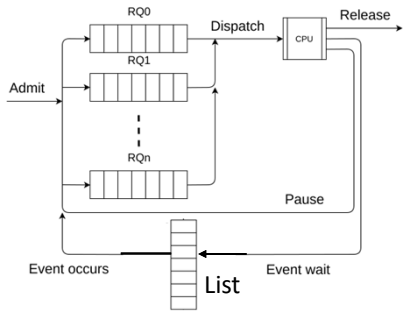
Simulation Results



21

Priority may be used in all scheduling policies

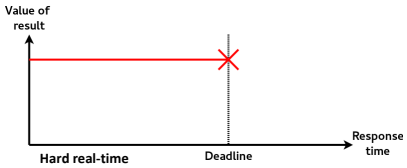
Empty first highest priority queue before starting on next queue

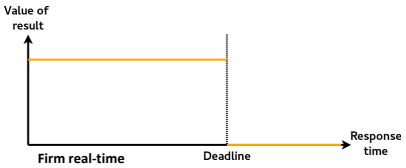


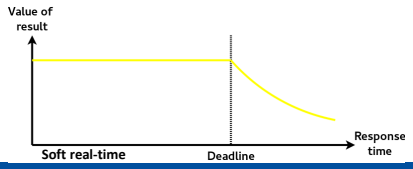
22

Real-time Scheduling

- In real-time scheduling, its all about the deadlines...
- Can your system be scheduled so all tasks can be performed without missing any deadlines??
- Back to some of the concept described in the introduction lecture...







Simple Task Model

- We start with a simple task model:

1. The application is assumed to consist of a fixed set of tasks
2. All tasks are periodic, with known periods
3. The tasks are completely independent of each other
4. All system's overheads, context-switching times and so on are ignored (i.e, assumed to have zero cost)
5. All tasks have a deadline equal to their period (that is, each task must complete before it is next released)
6. All tasks have a fixed worst-case execution time

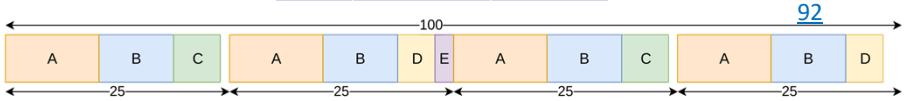
Task	Period (T)	Time (C)
A	25	10
B	25	8
C	50	5
D	50	4
E	100	2

Cyclic Executive

- Requires a fixed set of periodic tasks.
- Pre-plan and "hardcode" when each of the task should run
  - Static scheduling
- A major cycle is divided into several minor cycles
  - Each task has to be placed within these cycles.
  - No Processes/threads, is just a sequence of functions/procedure calls.
  - Easy to share data, concurrent access / race conditions not possible.

Task	Period (T)	Time (C)
A	25	10
B	25	8
C	50	5
D	50	4
E	100	2

(100/25)*10 = 40
(100/25)*8 = 32
(100/50)*5 = 10
(100/50)*4 = 8
(100/100)*2 = 2
92

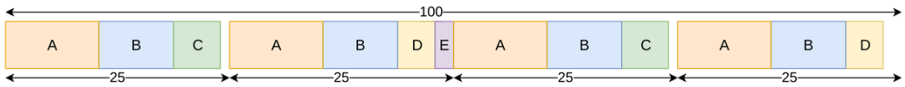


25

Cyclic Executive Problems

- Difficult to have sporadic tasks.
  (Sporadic task: Is performed when an event occur. There are limitations on how often the event can occur.)
- Difficult to have long tasks.
- Difficult to "construct" the cyclic executive.
- In larger systems, it is likely that there will be very large major cycles, with very many minor cycles.

Task	Period (T)	Time (C)
A	25	10
B	25	8
C	50	5
D	50	4
E	100	2



26

Fixed-Priority Scheduling (FPS)

Fixed-Priority Scheduling (FPS)

- This is the most widely used real-time scheduling approach.
  - If preemptive (which it probably should be), then the highest priority available task will always run.
- The priority of each task is found before run-time.
  - Priorities are not changed (static).
- Priorities should be assigned based on what is best for the whole system, not on the relative importance of the task.
- Optimal priority assignment for periodic, independent tasks is ***rate monotonic***.
  - The shorter the period, the higher priority.

Task	Period (T)	Time (C)	Priority (P)	
A	50	12	1	
B	40	10	2	
C	30	10	3	(Highest)

Is Scheduling Possible? Do we have enough CPU-time?

- Scheduling tests:
  - A sufficient test means that if the test is passed, the tasks are certain to meet their deadlines.
    (If the test fails, the tasks may still be able to meet their deadlines)
  - A necessary test means that if the test is failed, the tasks will miss their deadlines.
    (If a necessary test is passed doesn't guarantee that the task will meet deadline.)
  - An exact test is both necessary and sufficient.

- *Utilization-based scheduability test* is a simple test of fixed priority scheduling with rate monotonic.
  - Sufficient, but not necessary.
  - Require simple task model.

  - U = Utilization
  - N = Number of tasks
  - T_i = Period of task *i*
  - C_i = Computational time of task *i*

Simple task model:

a) The application is assumed to consist of a fixed set of tasks.
b) All tasks are periodic, with known periods.

Utilization criteria

$$U \equiv \sum_{i=1}^{N} \frac{C_i}{T_i} \leq N\,(2^{1/N} - 1)$$

complete before it is next released).
f) All tasks have a fixed worst-case execution time

Utilization limits

$$U \equiv \sum_{i=1}^{N} \frac{C_i}{T_i} \leq N\,(2^{1/N} - 1)$$

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

Utilization < 100 % for N >= 2

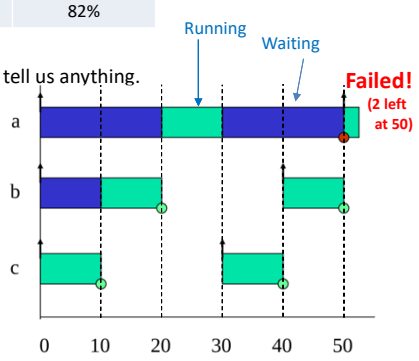
Example of Utilization (A)

- Use Utilization on previous example:

Task	Period (T)	Time (C)	Priority (P)	Utilization (U) = C/T
A	50	12	1	24%
B	40	10	2	25%
C	30	10	3	33%
			Sum	82%

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

- 82% is NOT below the limit for 3 tasks (78%)
  - In this case the utilization test does not really tell us anything.
- Time line can be used to draw the tasks, and evaluate if they are schedulable.
  - If all tasks start at time 0, it is sufficient if all tasks are able to make their first deadline.



31

Example of Utilization (B)

- Use Utilization on another example:

Task	Period (T)	Time (C)	Priority (P)	Utilization (U) = C/T
A	80	32	1	40%
B	40	5	2	12,5%
C	16	4	3	25%
			Sum	77,5%

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

- 77,5% is below the limit for 3 tasks (78%)
  - The utilization test guarantees that these tasks are scheduable.

32

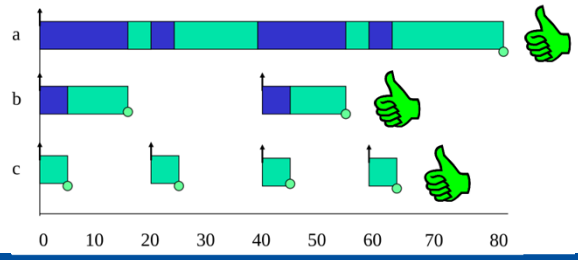
Example of Utilization (C)

- Use Utilization on the following example:

Task	Period (T)	Time (C)	Priority (P)	Utilization (U)
A	80	40	1	50%
B	40	10	2	25%
C	20	5	3	25%
			Sum	100%

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

- 100% is NOT below the limit for 3 tasks (78%)
  - In this case the utilization test does not really tell us anything.
- However….



Alternatives (improvements) for the Utilization Test 1 of 2

1. Tasks that have periods that are multiples of each other are considered to be in the same *family*. Counting *families of tasks* as tasks.
   - In a task set of 3 tasks, where 2 tasks are in the same family. Then the 2 task-limit (82,8%) instead of the 3 task-limit (78,0%) is used.

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

Example (task set C)

- All tasks are in family (20, 40, 80)
- Not necessary to use 3 task-limit (78.0%), instead the 1 task-limit (100%).

Task	Period (T)	Time (C)	Priority (P)	Utilization (U)
A	80	40	1	50%
B	40	10	2	25%
C	20	5	3	25%
			Sum	100%

Alternatives (improvements) for the Utilization Test 2 of 2

2. Alternative Formula:

$$\prod_{i=1}^{N}\left(\frac{C_i}{T_i}+1\right) \leq 2$$

Example:

Task	Period (T)	Time (C)	Priority (P)	Utilization (U) = C/T
A	76	32	1	42,1%
B	40	5	2	12,5%
C	16	4	3	25%
			Sum	79,6%

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

????

1,421 * 1,125 * 1,25 = 1.998 < 2

Criticism of Tests

$$U \equiv \sum_{i=1}^{N}\frac{C_i}{T_i} \leq N\left(2^{1/N}-1\right) \qquad \prod_{i=1}^{N}\left(\frac{C_i}{T_i}+1\right) \leq 2$$

- Not exact
- Not general
- BUT quite fast

The tests are sufficient but not necessary

Why should we use the two tests?

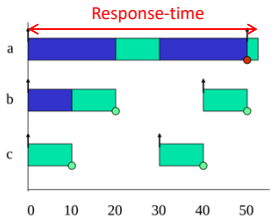
The first is simple to use

The second includes more situations than the first

Response-time Analysis 1 of 2

Response-time: The time from a task starts execution to it is completed [Burns and Wellings]
(The time from the submission of a request until the start of response. [Stallings])

- Is a method for calculating the **worst case response time** of a task, when there are higher priority tasks that interfere.
  - R_i : Response time of task i
  - D_i: Deadline of task i
  - C_i : The computational time of task i (Service time)
  - I_i : Interference from higher priority tasks



$$R_i \leq D_i$$

$$R_i = C_i + I_i$$

- If no interference, i.e. task$_i$ has highest priority / is the only task:

$$R_i = C_i$$

Response-time Analysis 2 of 2

- How can we calculate R_i??
  - C_i we already know
  - I_i is a bit more difficult
- This following test is **Exact** (sufficient and necessary)

Calculating R_i

- To calculate R_i, we need to get I_i.
  - We have to find out the maximum number of times EACH of the higher priority tasks can be released during R_i.
  - We come up with this formula, where T_j is the period of task j (which has higher priority than task i)

$$\text{Number of Releases} = \left\lceil \frac{R_i}{T_j} \right\rceil$$

(= Number of times task i is interrupted, rounded up)

- A release of task j will interfere with the computation of task i.
  - The computation time of task j determines the length of this interference.
  - The maximum interference time from task j on task will then be:

$$\left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

(= The time task j takes from task i)

Formula for Response Time Analysis

- The total interference from all higher priority tasks will then be:

$$I_i = \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

(= The time all tasks with higher priority takes from task i)

- We then have a formula for the response time of task i (R_i):

$$R_i = C_i + I_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

- Oh great... you need R_i to calculate R_i...

How to use the Formula

- The response time has to be calculated using a recurrence relationship.

$$w_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

- Suitable to solve with an algorithm:

```
for i in 1..N loop -- for each process in turn
  n := 0
  w_i^n := C_i
loop
    calculate new w_i^(n+1)
    if w_i^(n+1) = w_i^n then
      R_i = w_i^n
      exit value found
    end if
    if w_i^(n+1) > T_i then
      exit value not found
    end if
    n := n + 1
  end loop
end loop
```


Response Time Analysis on Paper

- You should not be surprised if you have to do this on the exam.
  (You should probably be surprised if you don't.)

Task	Period (T)	Time (C)	Priority (P)	Utilization (U)
A	7	3	3	42,9%
B	12	3	2	25,0%
C	20	5	1	25,0%
			Sum	92,9%

Number of Tasks	Utilization Limit
1	100,0%
2	82,8%
3	78,0%
4	75,7%
5	74,3%
10	71,8%

- Utilization analysis can not say whether this is scheduable or not.

Response Time Analysis on Paper

$$w_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

	T	C	P
A	7	3	3
B	12	3	2
C	20	5	1

- For Task A
  - Highest priority task, nothing can interfere.

$$R_a = 3 \quad \text{<= 7}$$

- For Task B
  - Start with the assumption that there is no interference.

$$w_b^0 = 3$$

  - Set w^0 into equation

$$w_b^1 = 3 + \left\lceil \frac{3}{7} \right\rceil 3 = 6$$

  - Set w^1 into equation

$$w_b^2 = 3 + \left\lceil \frac{6}{7} \right\rceil 3 = 6$$

  - No change in w, must be correct....

$$R_b = 6 \quad \text{<= 12}$$

43

Response Time Analysis on Paper

$$w_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

	T	C	P
A	7	3	3
B	12	3	2
C	20	5	1

- For Task C
  - Start with the assumption that there is no interference.
  - Set w^0 into equation
  - Set w^1 into equation
  - Set w^2 into equation
  - Set w^3 into equation
  - Set w^4 into equation
  - No change in w, must be correct....

	A	B
$w_c^0 = 5$		
$w_c^1 = 5 + \left\lceil \frac{5}{7} \right\rceil 3 + \left\lceil \frac{5}{12} \right\rceil 3 = 11$	$\left\lceil \frac{5}{7} \right\rceil 3$	$\left\lceil \frac{5}{12} \right\rceil 3$
$w_c^2 = 5 + \left\lceil \frac{11}{7} \right\rceil 3 + \left\lceil \frac{11}{12} \right\rceil 3 = 14$	$\left\lceil \frac{11}{7} \right\rceil 3$	$\left\lceil \frac{11}{12} \right\rceil 3$
$w_c^3 = 5 + \left\lceil \frac{14}{7} \right\rceil 3 + \left\lceil \frac{14}{12} \right\rceil 3 = 17$	$\left\lceil \frac{14}{7} \right\rceil 3$	$\left\lceil \frac{14}{12} \right\rceil 3$
$w_c^4 = 5 + \left\lceil \frac{17}{7} \right\rceil 3 + \left\lceil \frac{17}{12} \right\rceil 3 = 20$	$\left\lceil \frac{17}{7} \right\rceil 3$	$\left\lceil \frac{17}{12} \right\rceil 3$
$w_c^5 = 5 + \left\lceil \frac{20}{7} \right\rceil 3 + \left\lceil \frac{20}{12} \right\rceil 3 = 20$	$\left\lceil \frac{20}{7} \right\rceil 3$	$\left\lceil \frac{20}{12} \right\rceil 3$

$$R_c = 20 \quad \text{<= 20}$$

44

45

Endings

2023-02

Which of the following is a characteristic of the Rate Monotonic Scheduling algorithm? Select the ones that are true.

a. Dynamic priorities.

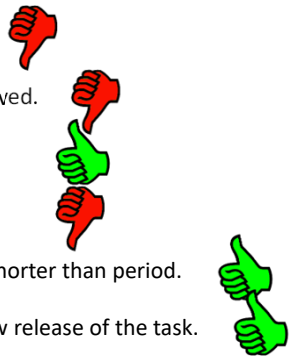
b. Priority inversion is inherently resolved.

c. Suitable for periodic tasks.

d. Utilizes deadline-based scheduling.

e. It is not suitable when deadline is shorter than period.

f. The deadline is the same as the new release of the task.



46